

Normalization, Functional Dependencies, Closure, Normal Forms, Anomalies, and Denormalization

These notes are written for DBMS placement preparation. They cover the theory, interview definitions, algorithms, examples, common traps, and quick revision points.

1. Why Normalization Matters

Normalization is the process of organizing data in a relational database to reduce redundancy and improve data integrity.

In simple words:

- Store each fact in one logical place.
- Avoid repeated data.
- Avoid update, insert, and delete anomalies.
- Design tables so that dependencies between attributes make sense.

Normalization is based on:

- Functional dependencies
- Candidate keys
- Prime and non-prime attributes
- Normal forms such as 1NF, 2NF, 3NF, and BCNF

Example of a badly designed table:

StudentID	StudentName	CourseID	CourseName	Instructor
1	Asha	C101	DBMS	Dr. Rao
1	Asha	C102	OS	Dr. Sen
2	Ravi	C101	DBMS	Dr. Rao

Problems:

- Student name is repeated.
- Course name is repeated.
- Instructor name is repeated.
- Updating **DBMS** instructor requires changing multiple rows.
- Cannot insert a new course unless at least one student enrolls.
- Deleting the last student in a course may delete course information.

Normalized design:

Student(StudentID, StudentName)

Course(CourseID, CourseName, Instructor)

`Enrollment(StudentID, CourseID)`

2. Important Terminology

Relation

A relation is a table.

Example:

`Student(RollNo, Name, Department, Email)`

Tuple

A tuple is a row in a table.

Attribute

An attribute is a column in a table.

Domain

The domain is the set of valid values an attribute can take.

Example:

`Age` may have integer values from 0 to 120.

Super Key

A super key is any set of attributes that can uniquely identify a tuple.

Example:

For `Student(RollNo, Email, Name)`, possible super keys:

- `{RollNo}`
- `{Email}`
- `{RollNo, Name}`
- `{Email, Name}`

Candidate Key

A candidate key is a minimal super key.

Minimal means no unnecessary attribute is included.

Example:

If both `RollNo` and `Email` uniquely identify a student, then:

- `{RollNo}` is a candidate key.
- `{Email}` is a candidate key.
- `{RollNo, Email}` is not a candidate key because it is not minimal.

Primary Key

A primary key is the candidate key chosen to identify tuples.

Alternate Key

Candidate keys not chosen as the primary key are alternate keys.

Prime Attribute

An attribute that is part of at least one candidate key is called a prime attribute.

Non-Prime Attribute

An attribute that is not part of any candidate key is called a non-prime attribute.

Example:

`Student(RollNo, Email, Name, Department)`

Candidate keys:

- `{RollNo}`
- `{Email}`

Prime attributes:

- `RollNo`
- `Email`

Non-prime attributes:

- `Name`
- `Department`

Composite Key

A key made of more than one attribute.

Example:

`Enrollment(StudentID, CourseID, Grade)`

Candidate key:

- `{StudentID, CourseID}`

Full Functional Dependency

An attribute is fully functionally dependent on a composite key if it depends on the whole key, not just part of it.

Example:

Enrollment(StudentID, CourseID, Grade)

{StudentID, CourseID} -> Grade

Here, Grade depends on both StudentID and CourseID.

Partial Dependency

A partial dependency exists when a non-prime attribute depends on only part of a composite candidate key.

Example:

Enrollment(StudentID, CourseID, StudentName, CourseName, Grade)

Candidate key:

{StudentID, CourseID}

Dependencies:

- StudentID -> StudentName
- CourseID -> CourseName
- {StudentID, CourseID} -> Grade

StudentName depends only on StudentID, not the full key. This is a partial dependency.

Transitive Dependency

A transitive dependency exists when:

A -> B and B -> C, so indirectly A -> C.

For normalization, the common issue is:

Primary key -> non-prime attribute -> another non-prime attribute

Example:

Student(RollNo, Name, DeptID, DeptName)

Dependencies:

- RollNo -> Name, DeptID
- DeptID -> DeptName

So:

RollNo -> DeptName transitively through DeptID.

3. Functional Dependency

A functional dependency describes a relationship between attributes in a relation.

If attribute set X determines attribute set Y, we write:

$X \rightarrow Y$

This means:

For any two rows, if the rows have the same value of X , then they must also have the same value of Y .

Example:

$RollNo \rightarrow StudentName$

If two rows have the same roll number, they must have the same student name.

Determinant

The left side of a functional dependency is called the determinant.

In $RollNo \rightarrow Name$, $RollNo$ is the determinant.

Dependent

The right side is the dependent.

In $RollNo \rightarrow Name$, $Name$ is the dependent.

Examples of Functional Dependencies

For:

$Student(RollNo, Name, Department, HOD)$

Possible dependencies:

- $RollNo \rightarrow Name$
- $RollNo \rightarrow Department$
- $Department \rightarrow HOD$
- $RollNo \rightarrow HOD$

The last one is transitive if $RollNo \rightarrow Department$ and $Department \rightarrow HOD$.

Functional Dependency Is About Semantics

Functional dependencies come from real-world rules, not just sample data.

If a table currently has:

City	State
Mumbai	Maharashtra
Pune	Maharashtra

You cannot conclude $State \rightarrow City$, because one state can have many cities.

But you may conclude $City \rightarrow State$ only if the business rule says each city belongs to one state.

Trivial Functional Dependency

A functional dependency $X \rightarrow Y$ is trivial if Y is a subset of X .

Examples:

- $\{A, B\} \rightarrow A$
- $\{A, B\} \rightarrow B$
- $\{A, B\} \rightarrow \{A, B\}$

These are always true.

Non-Trivial Functional Dependency

$X \rightarrow Y$ is non-trivial if Y is not a subset of X .

Example:

$RollNo \rightarrow Name$

Completely Non-Trivial Functional Dependency

$X \rightarrow Y$ is completely non-trivial if X and Y have no attributes in common.

Example:

$A \rightarrow B$

For $\{A, B\} \rightarrow \{B, C\}$, it is non-trivial but not completely non-trivial because B is common.

4. Armstrong's Axioms

Armstrong's axioms are rules used to infer all functional dependencies from a given set of dependencies.

1. Reflexivity

If Y is a subset of X , then:

$X \rightarrow Y$

Example:

$\{A, B\} \rightarrow A$

2. Augmentation

If:

$X \rightarrow Y$

Then:

$XZ \rightarrow YZ$

Example:

If $A \rightarrow B$, then:

$AC \rightarrow BC$

3. Transitivity

If:

$X \rightarrow Y$ and $Y \rightarrow Z$

Then:

$X \rightarrow Z$

Example:

If $RollNo \rightarrow DeptID$ and $DeptID \rightarrow DeptName$, then:

$RollNo \rightarrow DeptName$

Derived Rules

Union Rule

If:

$X \rightarrow Y$ and $X \rightarrow Z$

Then:

$X \rightarrow YZ$

Decomposition Rule

If:

$X \rightarrow YZ$

Then:

$X \rightarrow Y$ and $X \rightarrow Z$

Pseudo-Transitivity Rule

If:

$X \rightarrow Y$ and $WY \rightarrow Z$

Then:

$WX \rightarrow Z$

Example:

If $A \rightarrow B$ and $CB \rightarrow D$, then $AC \rightarrow D$.

5. Closure of Attribute Set

The closure of an attribute set X , written as X^+ , is the set of all attributes that can be functionally determined by X using the given functional dependencies.

Closure is used to:

- Find candidate keys.
- Check whether a functional dependency is valid.
- Test normal forms.
- Test lossless decomposition and dependency preservation.

Algorithm to Find Attribute Closure

Given relation R and functional dependencies F .

To find X^+ :

1. Start with $X^+ = X$.
2. Check each FD $A \rightarrow B$.
3. If A is a subset of X^+ , add B to X^+ .
4. Repeat until no more attributes can be added.

Example 1

Relation:

$R(A, B, C, D, E)$

Functional dependencies:

- $A \rightarrow B$
- $B \rightarrow C$
- $C \rightarrow D$
- $D \rightarrow E$

Find A^+ .

Start:

$A^+ = \{A\}$

Using $A \rightarrow B$:

$A^+ = \{A, B\}$

Using $B \rightarrow C$:

$A^+ = \{A, B, C\}$

Using $C \rightarrow D$:

$$A^+ = \{A, B, C, D\}$$

Using $D \rightarrow E$:

$$A^+ = \{A, B, C, D, E\}$$

So:

$$A^+ = \{A, B, C, D, E\}$$

Therefore, A is a candidate key if it is minimal.

Example 2

Relation:

$$R(A, B, C, D)$$

Functional dependencies:

- $A \rightarrow B$
- $B \rightarrow C$
- $C \rightarrow A$

Find A^+ .

$$A^+ = \{A\}$$

Use $A \rightarrow B$:

$$A^+ = \{A, B\}$$

Use $B \rightarrow C$:

$$A^+ = \{A, B, C\}$$

No dependency gives D .

So:

$$A^+ = \{A, B, C\}$$

A is not a super key for $R(A, B, C, D)$.

Find $\{A, D\}^+$.

Start:

$$\{A, D\}^+ = \{A, D\}$$

Use $A \rightarrow B$:

$$\{A, D\}^+ = \{A, B, D\}$$

Use $B \rightarrow C$:

$$\{A, D\}^+ = \{A, B, C, D\}$$

So $\{A, D\}$ is a super key.

Check minimality:

- $A^+ = \{A, B, C\}$, not all attributes.
- $D^+ = \{D\}$, not all attributes.

So $\{A, D\}$ is a candidate key.

6. Closure of Functional Dependency Set

The closure of a set of functional dependencies F , written as F^+ , is the set of all functional dependencies that can be inferred from F .

Usually in exams and interviews, you do not list all of F^+ because it can be very large. Instead, you use closure to check whether a specific FD follows from F .

To check whether $X \rightarrow Y$ follows from F :

1. Compute X^+ .
2. If Y is a subset of X^+ , then $X \rightarrow Y$ is implied by F .

Example:

$$F = \{A \rightarrow B, B \rightarrow C\}$$

Check if $A \rightarrow C$ is implied.

Find A^+ :

$$A^+ = \{A, B, C\}$$

Since C is in A^+ , $A \rightarrow C$ is implied.

7. Finding Candidate Keys

Candidate keys are minimal attribute sets whose closure contains all attributes of the relation.

Useful Rules

Attributes that never appear on the RHS of any FD must be part of every candidate key.

Attributes that appear only on the RHS are usually not necessary to start a key search, unless needed for minimality checks in special cases.

Attributes not present in any FD must be part of every candidate key.

Candidate Key Example

Relation:

$$R(A, B, C, D, E)$$

FDs:

- $A \rightarrow B$
- $BC \rightarrow D$
- $D \rightarrow E$
- $E \rightarrow C$

Find candidate keys.

Attributes:

A, B, C, D, E

RHS attributes:

B, D, E, C

Attribute not on RHS:

A

So every candidate key must contain A.

Find A^+ :

$A^+ = \{A, B\}$

Cannot proceed because $BC \rightarrow D$ needs C.

Try AC^+ :

Start:

$AC^+ = \{A, C\}$

Use $A \rightarrow B$:

$AC^+ = \{A, B, C\}$

Use $BC \rightarrow D$:

$AC^+ = \{A, B, C, D\}$

Use $D \rightarrow E$:

$AC^+ = \{A, B, C, D, E\}$

So AC is a key.

Try AD^+ :

Start:

$AD^+ = \{A, D\}$

Use $A \rightarrow B$:

$AD^+ = \{A, B, D\}$

Use $D \rightarrow E$:

$AD^+ = \{A, B, D, E\}$

Use $E \rightarrow C$:

$AD^+ = \{A, B, C, D, E\}$

So AD is a key.

Try AE^+ :

Start:

$AE^+ = \{A, E\}$

Use $A \rightarrow B$:

$AE^+ = \{A, B, E\}$

Use $E \rightarrow C$:

$AE^+ = \{A, B, C, E\}$

Use $BC \rightarrow D$:

$AE^+ = \{A, B, C, D, E\}$

So AE is a key.

Candidate keys:

- AC
- AD
- AE

Prime attributes:

- A
- C
- D
- E

Non-prime attribute:

- B

8. Minimal Cover or Canonical Cover

A minimal cover is a simplified equivalent set of functional dependencies.

It is useful for:

- 3NF decomposition
- Dependency preservation
- Removing redundant dependencies
- Database design questions

Properties of Minimal Cover

A minimal cover has:

1. Single attribute on RHS of every FD.
2. No extraneous attribute on LHS.
3. No redundant FD.

Steps to Find Minimal Cover

1. Split RHS attributes.
2. Remove extraneous attributes from LHS.
3. Remove redundant dependencies.

Example

Given:

$F = \{A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C\}$

Step 1: Split RHS.

$A \rightarrow B$

$A \rightarrow C$

$B \rightarrow C$

$A \rightarrow B$

$AB \rightarrow C$

Remove duplicate:

$A \rightarrow B$

$A \rightarrow C$

$B \rightarrow C$

$AB \rightarrow C$

Check if $A \rightarrow C$ is redundant:

Because $A \rightarrow B$ and $B \rightarrow C$, we get $A \rightarrow C$.

So remove $A \rightarrow C$.

Check $AB \rightarrow C$:

Since $B \rightarrow C$, $AB \rightarrow C$ is redundant.

Minimal cover:

$\{A \rightarrow B, B \rightarrow C\}$

9. Decomposition

Decomposition means splitting one relation into smaller relations.

Example:

$Student(rollNo, Name, DeptID, DeptName)$

Can be decomposed into:

$Student(rollNo, Name, DeptID)$

$Department(DeptID, DeptName)$

Goals of Decomposition

A good decomposition should be:

- Lossless
- Dependency preserving
- In a higher normal form

10. Lossless Decomposition

A decomposition is lossless if joining the decomposed tables gives back exactly the original table, without extra rows and without missing rows.

Binary Lossless Join Test

For relation R decomposed into R_1 and R_2 , the decomposition is lossless if:

$(R_1 \text{ intersection } R_2) \rightarrow R_1$

or

$(R_1 \text{ intersection } R_2) \rightarrow R_2$

That means the common attributes must be a super key in at least one decomposed relation.

Example

$R(A, B, C)$

FD:

$A \rightarrow B$

Decompose into:

$R1(A, B)$

$R2(A, C)$

Common attribute:

A

Check:

$A \rightarrow AB$

Since $A \rightarrow B$, A determines all attributes of $R1(A, B)$.

So the decomposition is lossless.

Lossy Decomposition Example

$R(A, B, C)$

No FD.

Decompose into:

$R1(A, B)$

$R2(B, C)$

Common attribute:

B

No dependency says $B \rightarrow A$ or $B \rightarrow C$.

So decomposition may be lossy.

11. Dependency Preservation

A decomposition is dependency preserving if all original functional dependencies can be checked by looking at individual decomposed tables, without needing to join them.

Why it matters:

- Constraints are easier to enforce.
- No expensive joins are needed for validation.
- Updates are safer.

Important interview point:

- 3NF decomposition can always be made lossless and dependency preserving.
- BCNF decomposition can always be made lossless, but may not preserve dependencies.

12. First Normal Form

A relation is in 1NF if:

- Every attribute contains atomic values.
- There are no repeating groups.
- Each cell has a single value.
- The order of rows and columns does not matter.

Not in 1NF Example

StudentID	Name	PhoneNumbers
1	Asha	9876, 9123
2	Ravi	8765

Problem:

PhoneNumbers contains multiple values in one cell.

Convert to 1NF

StudentID	Name	PhoneNumber
1	Asha	9876
1	Asha	9123
2	Ravi	8765

Better design:

Student(StudentID, Name)

StudentPhone(StudentID, PhoneNumber)

Common 1NF Violations

- Multiple phone numbers in one column.
- Comma-separated values.
- Repeating columns such as **Phone1**, **Phone2**, **Phone3**.
- Nested tables.
- Arrays stored in a column when relational atomicity is expected.

Interview Definition

1NF removes repeating groups and ensures atomic attribute values.

13. Second Normal Form

A relation is in 2NF if:

- It is in 1NF.
- It has no partial dependency of a non-prime attribute on a candidate key.

2NF mainly matters when the candidate key is composite.

If a relation has only single-attribute candidate keys, it is automatically in 2NF if it is already in 1NF.

2NF Violation Example

Relation:

Enrollment(StudentID, CourseID, StudentName, CourseName, Grade)

Candidate key:

{StudentID, CourseID}

FDs:

- StudentID -> StudentName
- CourseID -> CourseName
- {StudentID, CourseID} -> Grade

Problems:

- StudentName depends only on StudentID.
- CourseName depends only on CourseID.
- These are partial dependencies.

Convert to 2NF

Student(StudentID, StudentName)

Course(CourseID, CourseName)

Enrollment(StudentID, CourseID, Grade)

Now:

- Student details depend on student key.
- Course details depend on course key.
- Grade depends on the full enrollment key.

What 2NF Removes

2NF removes partial dependency.

What 2NF Does Not Remove

2NF does not necessarily remove transitive dependency.

Example:

Student(RollNo, Name, DeptID, DeptName)

Candidate key:

RollNo

No partial dependency because key is not composite.

But:

$\text{RollNo} \rightarrow \text{DeptID}$

$\text{DeptID} \rightarrow \text{DeptName}$

This is transitive dependency, so the table may still violate 3NF.

14. Third Normal Form

A relation is in 3NF if:

- It is in 2NF.
- It has no transitive dependency of non-prime attributes on candidate keys.

Formal definition:

For every non-trivial FD $X \rightarrow A$, at least one of these must be true:

- X is a super key.
- A is a prime attribute.

3NF Violation Example

Relation:

$\text{Student}(\text{RollNo}, \text{Name}, \text{DeptID}, \text{DeptName})$

FDs:

- $\text{RollNo} \rightarrow \text{Name}, \text{DeptID}$
- $\text{DeptID} \rightarrow \text{DeptName}$

Candidate key:

RollNo

Here:

$\text{RollNo} \rightarrow \text{DeptID} \rightarrow \text{DeptName}$

DeptName depends on RollNo through DeptID .

This is a transitive dependency.

Convert to 3NF

$\text{Student}(\text{RollNo}, \text{Name}, \text{DeptID})$

$\text{Department}(\text{DeptID}, \text{DeptName})$

Why Prime Attribute Exception Exists

3NF allows $X \rightarrow A$ if A is prime, even when X is not a super key.

This makes 3NF less strict than BCNF and helps preserve dependencies.

3NF Example With Prime Attribute Exception

Relation:

$R(A, B, C)$

FDs:

- $AB \rightarrow C$
- $C \rightarrow B$

Candidate keys:

- AB
- AC

Prime attributes:

- A
- B
- C

For $C \rightarrow B$:

- C is not a super key.
- But B is prime.

So the relation can be in 3NF.

But it is not in BCNF because determinant C is not a super key.

What 3NF Removes

3NF removes:

- Partial dependency
- Transitive dependency involving non-prime attributes

Interview Definition

3NF says every non-prime attribute must depend only on candidate keys, not on other non-prime attributes.

15. Boyce-Codd Normal Form

BCNF is a stricter version of 3NF.

A relation is in BCNF if:

For every non-trivial functional dependency $X \rightarrow Y$, X must be a super key.

Difference Between 3NF and BCNF

3NF condition for $X \rightarrow A$:

- X is a super key, or
- A is prime

BCNF condition:

- X must be a super key

So every BCNF relation is in 3NF, but every 3NF relation is not necessarily in BCNF.

BCNF Violation Example

Relation:

$R(\text{Student}, \text{Course}, \text{Instructor})$

Business rules:

- A student can take many courses.
- Each course can have many students.
- Each instructor teaches only one course.
- Each course has one instructor.

FDs:

- $\{\text{Student}, \text{Course}\} \rightarrow \text{Instructor}$
- $\text{Instructor} \rightarrow \text{Course}$

Candidate keys:

- $\{\text{Student}, \text{Course}\}$
- $\{\text{Student}, \text{Instructor}\}$

Check FD:

$\text{Instructor} \rightarrow \text{Course}$

Instructor is not a super key because $\text{Instructor}^+ = \{\text{Instructor}, \text{Course}\}$, not Student.

So BCNF is violated.

It may still be in 3NF because Course is prime.

Decompose to BCNF

Original:

$R(\text{Student}, \text{Course}, \text{Instructor})$

Decompose using $\text{Instructor} \rightarrow \text{Course}$:

$R_1(\text{Instructor}, \text{Course})$

$R_2(\text{Student}, \text{Instructor})$

Now:

- In R_1 , Instructor is a key.
- In R_2 , no problematic dependency remains.

Important BCNF Facts

- BCNF removes more redundancy than 3NF.
- BCNF decomposition is always lossless.
- BCNF decomposition may not preserve all dependencies.
- 3NF is sometimes preferred in practical design because it can preserve dependencies.

16. Higher Normal Forms Briefly

Placements usually focus on 1NF, 2NF, 3NF, and BCNF, but you may be asked about higher forms.

Fourth Normal Form

4NF deals with multivalued dependencies.

A relation is in 4NF if:

- It is in BCNF.
- It has no non-trivial multivalued dependency except those where the determinant is a super key.

Example:

$\text{Student}(\text{StudentID}, \text{Skill}, \text{Hobby})$

If skills and hobbies are independent:

- $\text{StudentID} \twoheadrightarrow \text{Skill}$
- $\text{StudentID} \twoheadrightarrow \text{Hobby}$

This causes combinations of every skill with every hobby.

Better:

$\text{StudentSkill}(\text{StudentID}, \text{Skill})$

$\text{StudentHobby}(\text{StudentID}, \text{Hobby})$

Fifth Normal Form

5NF deals with join dependencies.

It ensures a relation cannot be further decomposed without losing information.

It is rarely asked deeply in fresher interviews.

17. Anomalies

Anomalies are problems caused by redundant or poorly organized data.

There are three major anomalies:

- Insertion anomaly
- Update anomaly
- Deletion anomaly

Example Table

`StudentCourse(StudentID, StudentName, CourseID, CourseName, Instructor)`

StudentID	StudentName	CourseID	CourseName	Instructor
1	Asha	C101	DBMS	Dr. Rao
1	Asha	C102	OS	Dr. Sen
2	Ravi	C101	DBMS	Dr. Rao

Insertion Anomaly

An insertion anomaly occurs when you cannot insert a fact without inserting some unrelated fact.

Example:

You cannot insert a new course `C103, Networks, Dr. Mehta` unless some student enrolls in it, because the table requires `StudentID` and `StudentName`.

Update Anomaly

An update anomaly occurs when the same fact is stored in multiple places and all copies must be updated.

Example:

If the instructor for DBMS changes from `Dr. Rao` to `Dr. Iyer`, every row with `C101` must be updated.

If one row is missed, the database becomes inconsistent.

Deletion Anomaly

A deletion anomaly occurs when deleting one fact unintentionally deletes another important fact.

Example:

If Ravi is the last student enrolled in `C101`, deleting Ravi's enrollment may also delete the only stored information about the DBMS course and its instructor.

How Normalization Helps

Normalize into:

`Student(StudentID, StudentName)`

`Course(CourseID, CourseName, Instructor)`

`Enrollment(StudentID, CourseID)`

Now:

- You can insert a course without a student.
- Updating instructor happens in one row.
- Deleting enrollment does not delete course information.

18. Denormalization

Denormalization is the deliberate process of adding redundancy to improve read performance.

It is usually done after normalization, not instead of understanding normalization.

Why Denormalize?

Normalized databases reduce redundancy, but queries may need many joins.

Denormalization can help when:

- Read performance is more important than write simplicity.
- Reports need precomputed values.
- Joins are too expensive.
- Data warehouses need faster analytical queries.
- Applications need low-latency reads.

Denormalization Examples

Storing Derived Values

Instead of calculating order total every time:

`Order(OrderID, CustomerID, TotalAmount)`

But `TotalAmount` can be derived from order items.

Risk:

If order items change and total is not updated, data becomes inconsistent.

Duplicating Frequently Needed Data

`Order(OrderID, CustomerID, CustomerName, OrderDate)`

`CustomerName` is already in the `Customer` table, but duplicating it can avoid a join in reports.

Risk:

If customer name changes, old orders may become inconsistent unless history is intended.

Summary Tables

`DailySalesSummary(Date, TotalOrders, TotalRevenue)`

This avoids scanning millions of orders repeatedly.

Risk:

Summary must be refreshed correctly.

Normalization vs Denormalization

Normalization	Denormalization
Reduces redundancy	Adds controlled redundancy
Improves data integrity	Improves read performance
More tables	Fewer joins in some queries
Better for OLTP writes	Common in analytics/reporting
Avoids anomalies	Can introduce anomalies

When to Denormalize

Denormalize only when:

- You have measured performance problems.
- Queries are read-heavy.
- Joins are expensive.
- The application can maintain consistency.
- The redundancy is intentional and documented.

Interview Answer

Denormalization is a performance optimization technique where redundant data is intentionally introduced into a normalized database to reduce joins and speed up read queries. It improves read performance but may increase storage, update complexity, and risk of inconsistency.

19. Normal Form Comparison Table

Normal Form	Main Requirement	Removes
1NF	Atomic values, no repeating groups	Multi-valued attributes
2NF	1NF plus no partial dependency	Partial dependency
3NF	2NF plus no transitive dependency of non-prime attributes	Transitive dependency
BCNF	Every determinant is a super key	More redundancy than 3NF
4NF	BCNF plus no problematic multivalued dependency	Independent multi-valued facts

Normal Form	Main Requirement	Removes
5NF	No problematic join dependency	Complex join redundancy

20. Dependency Rules for Normal Forms

2NF Check

Ask:

Does any non-prime attribute depend on part of a composite candidate key?

If yes, not in 2NF.

3NF Check

For every FD $X \rightarrow A$, check:

- Is X a super key?
- Or is A prime?

If neither, violates 3NF.

BCNF Check

For every FD $X \rightarrow Y$, check:

- Is X a super key?

If no, violates BCNF.

21. Step-by-Step Normalization Strategy

Given a relation and FDs:

1. Identify all attributes.
2. Find candidate keys using closure.
3. Identify prime and non-prime attributes.
4. Check 1NF.
5. Check for partial dependencies to test 2NF.
6. Check for transitive dependencies or use formal 3NF condition.
7. Check BCNF by testing whether every determinant is a super key.
8. If violation exists, decompose.
9. Check if decomposition is lossless.
10. Check if dependencies are preserved.

22. Common Placement Questions

Question 1: What is normalization?

Normalization is the process of organizing relational database tables to reduce redundancy and avoid insertion, update, and deletion anomalies. It decomposes large tables into smaller, well-structured tables based on functional dependencies.

Question 2: What is functional dependency?

A functional dependency $X \rightarrow Y$ means that the value of attribute set X uniquely determines the value of attribute set Y . If two tuples have the same X value, they must have the same Y value.

Question 3: What is the difference between 2NF and 3NF?

2NF removes partial dependency of non-prime attributes on a composite key. 3NF removes transitive dependency of non-prime attributes on candidate keys.

Question 4: What is the difference between 3NF and BCNF?

BCNF is stricter than 3NF. In 3NF, for every FD $X \rightarrow A$, either X must be a super key or A must be prime. In BCNF, X must always be a super key for every non-trivial FD.

Question 5: Is every BCNF relation in 3NF?

Yes. Every BCNF relation is in 3NF because BCNF requires every determinant to be a super key, which satisfies the 3NF condition.

Question 6: Is every 3NF relation in BCNF?

No. A 3NF relation may violate BCNF when the RHS attribute is prime but the LHS is not a super key.

Question 7: What is closure?

The closure of an attribute set X , written X^+ , is the set of all attributes that can be determined from X using the given functional dependencies.

Question 8: Why is closure useful?

Closure is used to find candidate keys, check whether a functional dependency is implied, and test normal forms.

Question 9: What are anomalies?

Anomalies are problems caused by redundant data. They include insertion anomaly, update anomaly, and deletion anomaly.

Question 10: What is denormalization?

Denormalization is the intentional introduction of redundancy into a database design to improve read performance. It can reduce joins but increases storage and risk of inconsistency.

23. Worked Example: Normalize to 3NF

Relation:

$R(\text{StudentID}, \text{StudentName}, \text{CourseID}, \text{CourseName}, \text{Instructor}, \text{Grade})$

FDs:

- $\text{StudentID} \rightarrow \text{StudentName}$
- $\text{CourseID} \rightarrow \text{CourseName}, \text{Instructor}$
- $\{\text{StudentID}, \text{CourseID}\} \rightarrow \text{Grade}$

Candidate key:

$\{\text{StudentID}, \text{CourseID}\}$

1NF

Assume all values are atomic.

So relation is in 1NF.

2NF Check

Candidate key is composite.

Partial dependencies:

- $\text{StudentID} \rightarrow \text{StudentName}$
- $\text{CourseID} \rightarrow \text{CourseName}, \text{Instructor}$

So relation is not in 2NF.

Decompose to 2NF

$\text{Student}(\text{StudentID}, \text{StudentName})$

$\text{Course}(\text{CourseID}, \text{CourseName}, \text{Instructor})$

$\text{Enrollment}(\text{StudentID}, \text{CourseID}, \text{Grade})$

3NF Check

Check each table:

$\text{Student}(\text{StudentID}, \text{StudentName})$

- StudentID is key.
- In 3NF.

$\text{Course}(\text{CourseID}, \text{CourseName}, \text{Instructor})$

- CourseID is key.
- In 3NF if no dependency like $\text{Instructor} \rightarrow \text{CourseName}$ exists.

$\text{Enrollment}(\text{StudentID}, \text{CourseID}, \text{Grade})$

- $\{\text{StudentID}, \text{CourseID}\}$ is key.

- In 3NF.

Final 3NF design:

- `Student(StudentID, StudentName)`
- `Course(CourseID, CourseName, Instructor)`
- `Enrollment(StudentID, CourseID, Grade)`

24. Worked Example: 3NF but Not BCNF

Relation:

$R(A, B, C)$

FDs:

- $AB \rightarrow C$
- $C \rightarrow B$

Find candidate keys.

AB^+ :

$AB^+ = \{A, B, C\}$

So AB is a key.

AC^+ :

Start:

$AC^+ = \{A, C\}$

Use $C \rightarrow B$:

$AC^+ = \{A, B, C\}$

So AC is a key.

Candidate keys:

- AB
- AC

Prime attributes:

- A
- B
- C

Check 3NF:

FD $AB \rightarrow C$:

- AB is a super key.

- OK.

FD $C \rightarrow B$:

- C is not a super key.
- But B is prime.
- OK for 3NF.

Check BCNF:

FD $C \rightarrow B$:

- C is not a super key.
- Violates BCNF.

Therefore:

R is in 3NF but not BCNF.

25. Worked Example: BCNF Decomposition

Relation:

$R(A, B, C)$

FDs:

- $A \rightarrow B$
- $B \rightarrow C$

Candidate key:

A

Check BCNF:

$A \rightarrow B$:

- A is a key.
- OK.

$B \rightarrow C$:

- B is not a key.
- Violates BCNF.

Decompose using $B \rightarrow C$:

$R_1(B, C)$

$R_2(A, B)$

Check:

$R_1(B, C)$:

- $B \rightarrow C$
- B is key.
- BCNF.

$R2(A, B)$:

- $A \rightarrow B$
- A is key.
- BCNF.

The decomposition is lossless because common attribute B determines $R1(B, C)$.

26. Quick Tricks for Exams and Interviews

- Always find candidate keys first.
- Do not decide 2NF, 3NF, or BCNF without knowing prime attributes.
- 2NF is only interesting when a candidate key is composite.
- For 3NF, RHS being prime can save the dependency.
- For BCNF, RHS being prime does not matter.
- BCNF is stricter than 3NF.
- 3NF can preserve dependencies; BCNF may not.
- Lossless decomposition is mandatory.
- Dependency preservation is desirable but not always possible with BCNF.
- Redundancy causes anomalies.
- Denormalization is controlled redundancy for performance.

27. Common Mistakes

Mistake 1: Confusing Super Key and Candidate Key

Every candidate key is a super key, but every super key is not a candidate key.

$\{RollNo\}$ may be a candidate key.

$\{RollNo, Name\}$ is a super key but not a candidate key because it is not minimal.

Mistake 2: Thinking Primary Key Is the Only Candidate Key

A relation can have multiple candidate keys.

Only one is selected as the primary key.

Mistake 3: Checking Normal Forms Using Only Primary Key

Normal forms are based on all candidate keys, not just the primary key.

Mistake 4: Saying 3NF Means No Transitive Dependency at All

The formal 3NF rule allows some dependencies if the RHS is a prime attribute.

Mistake 5: Assuming BCNF Always Preserves Dependencies

BCNF decomposition is lossless, but dependency preservation is not guaranteed.

Mistake 6: Inferring FDs Only From Current Data

Functional dependencies must come from business rules, not just sample rows.

28. One-Page Revision

Functional dependency:

$X \rightarrow Y$ means X determines Y .

Closure:

X^+ is everything determined by X .

Candidate key:

Minimal attribute set whose closure contains all attributes.

Prime attribute:

Attribute that belongs to at least one candidate key.

1NF:

Atomic values, no repeating groups.

2NF:

No partial dependency of non-prime attributes on a candidate key.

3NF:

For every FD $X \rightarrow A$, X is a super key or A is prime.

BCNF:

For every FD $X \rightarrow Y$, X is a super key.

Anomalies:

- Insert anomaly: cannot insert one fact without another.
- Update anomaly: same fact updated in many rows.
- Delete anomaly: deleting one fact removes another fact accidentally.

Denormalization:

Intentional redundancy to improve read performance.

29. Practice Problems

Problem 1

$R(A, B, C, D)$

FDs:

- $A \rightarrow B$
- $B \rightarrow C$
- $C \rightarrow D$

Tasks:

- Find A^+ .
- Find candidate keys.
- Check BCNF.

Answer:

$A^+ = \{A, B, C, D\}$

Candidate key:

A

BCNF:

$A \rightarrow B$ is OK because A is key.

$B \rightarrow C$ violates BCNF because B is not a super key.

$C \rightarrow D$ violates BCNF because C is not a super key.

Problem 2

$R(A, B, C)$

FDs:

- $A \rightarrow B$
- $B \rightarrow A$
- $B \rightarrow C$

Find candidate keys.

Answer:

A^+ :

$A \rightarrow B$, then $B \rightarrow C$, so $A^+ = \{A, B, C\}$.

B^+ :

$B \rightarrow A$ and $B \rightarrow C$, so $B^+ = \{A, B, C\}$.

Candidate keys:

- A
- B

Prime attributes:

- A
- B

Non-prime attribute:

- C

Problem 3

$R(A, B, C)$

FDs:

- $A \rightarrow B$
- $B \rightarrow C$

Is it in 3NF?

Answer:

Candidate key:

A

Prime attribute:

A

FD $A \rightarrow B$:

- A is super key.
- OK.

FD $B \rightarrow C$:

- B is not super key.
- C is not prime.
- Violates 3NF.

So relation is not in 3NF.

30. Final Interview Summary

Normalization is a database design technique that structures tables to reduce redundancy and prevent anomalies. Functional dependencies define how attributes determine each other. Attribute closure helps find candidate keys and test dependencies. 1NF ensures atomic values, 2NF removes partial dependency, 3NF removes problematic transitive dependency, and BCNF requires every determinant to be a super key. Denormalization is the opposite direction: controlled redundancy for faster reads, used carefully when performance demands it.